

1. ZADATAK

PRII – G1

Izvršiti definiciju funkcija na način koji odgovara opisu (komentarima) datom neposredno uz pozive ili nazive funkcija. Možete dati komentar na bilo koju liniju code-a koju smatrate da bi trebalo unaprijediti ili da će eventualno uzrokovati grešku prilikom kompajliranja. Također, možete dodati dodatne funkcije ili metode koje će vam olakšati implementaciju programa.

```
#include <iostream>
using namespace std;

string crt = "\n-----\n";

string PORUKA_TELEFON = crt +
"TELEFONE ISKLJUCITE I ODLOZITE U TORBU, DZEP ILI DRUGU LOKACIJU VAN
DOHVATA.\n"
"CESTO SE NA TELEFONIMA (PRO)NALAZE PROGRAMSKI KODOVI KOJI MOGU BITI
ISKORISTENI ZA\n"
"RJESAVANJE ISPITNOG ZADATKA, STO CE, U SLUCAJU PRONALASKA, BITI
SANKCIONISANO." + crt;

string PORUKA_ISPIT = crt +
"0. PROVJERITE DA LI ZADACI PRIPADAJU VASOJ GRUPI (G1/G2)\n"
"1. SVE KLASA SA DINAMICKOM ALOKACIJOM MORAJU IMATI ISPRAVAN
DESTRUKTOR\n"
"2. IZOSTAVLJANJE DESTRUKTORA ILI NJEGOVIH DIJELOVA BIT CE OZNACENO
KAO TM\n"
"3. ATRIBUTI, METODE I PARAMETRI MORAJU BITI IDENTICNI ONIMA U TESTNOJ
MAIN FUNKCIJI,\n"
"    OSIM AKO POSTOJI JASNO OPISAN RAZLOG ZA MODIFIKACIJU\n"
"4. IZUZETKE BACAJTE SAMO TAMO GDJE JE IZRICITO NAGLASENO\n"
"5. SVE METODE KOJE SE POZIVAJU U MAIN-U MORAJU POSTOJATI.\n"
"    AKO NEMATE ZELJENU IMPLEMENTACIJU, OSTAVITE PRAZNO TIJELO ILI
VRATITE DEFAULT VRIJEDNOST\n"
"6. U MAIN FUNKCIJI MOZETE DODAVATI TESTNE PODATKE I POZIVE PO
VLASTITOM IZBORU\n"
"7. TESTIRAJTE PROGRAM U OBA MODA (F5 i Ctrl+F5)" + crt;

char* AlocirajTekst(const char* tekst) {
    if (!tekst) return nullptr;
    size_t vel = strlen(tekst) + 1;
    char* temp = new char[vel];
    strcpy_s(temp, vel, tekst);
    return temp;
}

string GenerisiSifru(const char* imePrezime, int redniBroj);
bool ValidirajSifru(const string& sifra);

enum TipProstora { RADNO_MJESTO, SALA, STUDIO, LABORATORIJA };
const char* TipProstoraNazivi[] = { "RADNO MJESTO", "SALA", "STUDIO",
"LABORATORIJA" };

template<class T1, class T2, int max>
class Kolekcija {
    T1 _prvi[max];
```

```

    T2 _drugi[max];
    int* _trenutno;
public:
    Kolekcija() { _trenutno = new int(0); }
    ~Kolekcija() { delete _trenutno; _trenutno = nullptr; }
    int GetTrenutno() const { return *_trenutno; }
    T1& GetPrvi(int indeks) { return _prvi[indeks]; }
    T2& GetDrugi(int indeks) { return _drugi[indeks]; }
    T1& operator[](int indeks) { return _prvi[indeks]; }
    friend ostream& operator<<(ostream& COUT, Kolekcija& obj) {
        for (int i = 0; i < obj.GetTrenutno(); i++)
            COUT << obj.GetPrvi(i) << " " << obj.GetDrugi(i) << endl;
        return COUT;
    }
};

class DatumVrijeme {
    int* _godina, * _mjesec, * _dan, * _sati, * _minute;
public:
    DatumVrijeme(int dan = 1, int mjesec = 1, int godina = 2000, int
sati = 0, int minute = 0) {
        _godina = new int(godina);
        _mjesec = new int(mjesec);
        _dan = new int(dan);
        _sati = new int(sati);
        _minute = new int(minute);
    }
    ~DatumVrijeme() {
        delete _godina; delete _mjesec; delete _dan;
        delete _sati; delete _minute;
    }
    friend ostream& operator<<(ostream& COUT, DatumVrijeme & obj) {
        COUT << *obj._dan << "." << *obj._mjesec << "." <<
*obj._godina << " " << *obj._sati << ":" << *obj._minute;
        return COUT;
    }
};

class Rezervacija {
    char* _oznaka;
    TipProstora _tipProstora;
    DatumVrijeme _pocetak;
    int _trajanjeMinuta;
public:
    Rezervacija(const char* oznaka, TipProstora tipProstora,
DatumVrijeme pocetak, int trajanjeMinuta)
        : _tipProstora(tipProstora), _pocetak(pocetak),
_trajanjeMinuta(trajanjeMinuta) {
        _oznaka = AlocirajTekst(oznaka);
    }
    ~Rezervacija() { delete[] _oznaka; _oznaka = nullptr; }
    const char* GetOznaka() const { return _oznaka; }
    TipProstora GetTipProstora() const { return _tipProstora; }
    DatumVrijeme& GetPocetak() { return _pocetak; }
    int GetTrajanjeMinuta() const { return _trajanjeMinuta; }
};

class Korisnik {
    static int _id;

```

```

    char* _sifra;
    char* _imePrezime;
    vector<Rezervacija> _rezervacije;
public:
    Korisnik(const char* imePrezime = "") {
        _imePrezime = AlocirajTekst(imePrezime);
        _sifra = AlocirajTekst(GenerisiSifru(imePrezime,
_id).c_str());
        _id++;
    }
    ~Korisnik() {
        delete[] _sifra; _sifra = nullptr;
        delete[] _imePrezime; _imePrezime = nullptr;
    }
    const char* GetSifra() const { return _sifra; }
    const char* GetImePrezime() const { return _imePrezime; }
    vector<Rezervacija>& GetRezervacije() { return _rezervacije; }
    friend ostream& operator<<(ostream& COUT, Korisnik& obj) {
        COUT << obj._imePrezime << " [" << obj._sifra << "]" << endl;
        for (auto& rezervacija : obj._rezervacije)
            //ToString metoda klase Rezervacija vraca podatke o
rezervaciji u formatu:
                //10.09.2026 09:00 SALA-A SALA 60 min
                COUT << " - " << rezervacija.ToString() << endl;
        return COUT;
    }
};

class CentarZaRad {
    char* _naziv;
    vector<Korisnik> _korisnici;
public:
    CentarZaRad(const char* naziv) { _naziv = AlocirajTekst(naziv); }
    ~CentarZaRad() { delete[] _naziv; _naziv = nullptr; }
    CentarZaRad(const CentarZaRad& obj) {
        _naziv = AlocirajTekst(obj._naziv);
        _korisnici = obj._korisnici;
    }
    const char* GetNaziv() const { return _naziv; }
    vector<Korisnik>& GetKorisnici() { return _korisnici; }
};

const char* GetOdgovorNaPrvoPitanje() {
    cout << "Pitanje -> Pojasnite razliku izmedju koristenja kljucne
rijeci abstract i cistih virtualnih metoda?\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

const char* GetOdgovorNaDrugoPitanje() {
    cout << "Pitanje -> Pojasnite korelaciju izmedju polimorfizma i
virtualnih metoda, te zbog cega se javlja potreba za virtualnim
destruktorom?\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

int main() {
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_ISPIT; cin.get(); system("cls");

```

```
cout << GetOdgovorNaPrvoPitanje() << crt;
cin.get();
cout << GetOdgovorNaDrugoPitanje() << crt;
cin.get();

//funkcija generise sifru korisnika na osnovu imena i prezimena,
rednog broja i trenutne godine.
//sifra je u formatu CW-IN-BBB/GGGG, gdje IN predstavlja
inicijale, BBB redni broj korisnika
//popunjen nulama na slobodnim mjestima, a GGGG trenutnu godinu
dobijenu iz sistema.
//funkciju koristiti u konstruktoru klase Korisnik za
inicijalizaciju atributa _sifra.
if (GenerisiSifru("Amina Buric", 3) == "CW-AB-003/2026")
    cout << "Sifra OK" << crt;
if (GenerisiSifru("Amar Macic", 15) == "CW-AM-015/2026")
    cout << "Sifra OK" << crt;
if (GenerisiSifru("Maid Ramic", 156) == "CW-MR-156/2026")
    cout << "Sifra OK" << crt;

//koristeci regex, funkcija ValidirajSifru provjerava da li je
sifra zapisana u prethodno
//definisanom formatu. funkcija vraca true ako je sifra validna, u
suprotnom vraca false.
if (ValidirajSifru("CW-AB-003/2026"))
    cout << "SIFRA VALIDNA" << crt;
if (!ValidirajSifru("CW-Ab-003/2026") && !ValidirajSifru("CW-AB-
03/2026") &&
    !ValidirajSifru("CW/AB-003-2026"))
    cout << "SIFRA NIJE VALIDNA" << crt;

Kolekcija<int, string, 20> termini;
for (int i = 0; i < 8; i++)
    termini.Dodaj(i, "Termin_" + to_string(i));
cout << termini << crt;

//DodajNaPoziciju dodaje novi par na lokaciju definisanu prvim
parametrom, pomjera postojece
//elemente udesno i vraca trenutno stanje kolekcije. u slucaju
popunjene kolekcije ili
//neispravne lokacije potrebno je baciti izuzetak.
Kolekcija<int, string, 20> prosireniTermini =
termini.DodajNaPoziciju(2, 99, "Poseban termin");
cout << prosireniTermini << crt;

//UkloniRaspon od lokacije definisane prvim parametrom uklanja
broj elemenata definisan
//drugim parametrom, a vraca pokazivac na novu kolekciju koja
sadrzi uklonjene elemente.
//pozivalac je odgovoran za dealokaciju vracene kolekcije.
Kolekcija<int, string, 20>* uklonjeniTermini =
prosireniTermini.UkloniRaspon(3, 2);
cout << "Uklonjeni elementi:" << crt << *uklonjeniTermini;
cout << "Preostali elementi:" << crt << prosireniTermini;
delete uklonjeniTermini;

try {
    //za neispravan raspon potrebno je baciti izuzetak
```

```
        termini.UkloniRaspon(6, 5);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    DatumVrijeme vrijeme1(10, 9, 2026, 9, 0), vrijeme2(10, 9, 2026, 9,
30),
        vrijeme3(10, 9, 2026, 10, 0), vrijeme4(10, 9, 2026, 12, 0);

    Rezervacija salaA("SALA-A", SALA, vrijeme1, 60);
    Rezervacija studio1("STUDIO-1", STUDIO, vrijeme2, 90);
    Rezervacija radnoMjesto("RM-12", RADNO_MJESTO, vrijeme3, 60);
    Rezervacija laboratorija("LAB-1", LABORATORIJA, vrijeme4, 400);

    //ToString metoda vraca podatke o rezervaciji u formatu prikazanom
u nastavku.
    //voditi racuna o prikazu jednocifrenih vrijednosti datuma i
vremena (npr. 9 -> 09).
    cout << salaA.ToString() << crt;
    //10.09.2026 09:00 SALA-A SALA 60 min

    //ImaKonfliktSa vraca true ako su rezervacije istog datuma i
njihovi vremenski intervali
//se preklapaju. rezervacija koja pocinje u trenutku kada
prethodna završava nije konfliktna.
    if (salaA.ImaKonfliktSa(studio1))
        cout << "Termini se preklapaju" << crt;
    if (!salaA.ImaKonfliktSa(radnoMjesto))
        cout << "Termini se ne preklapaju" << crt;

    Korisnik amina("Amina Buric"), goran("Goran Skondric"),
berun("Berun Agic");

    //DodajRezervaciju dodaje rezervaciju korisniku ako se ona ne
preklapa sa nekom od ranije
//dodanih rezervacija i ako ukupno trajanje svih rezervacija
korisnika u jednom danu ne
//prelazi 480 minuta. metoda vraca true ako je rezervacija dodana,
u suprotnom vraca false.
    if (amina.DodajRezervaciju(salaA))
        cout << "Rezervacija dodana" << crt;
    if (!amina.DodajRezervaciju(studio1))
        cout << "Rezervacija nije dodana - preklapanje termina" <<
crt;
    if (amina.DodajRezervaciju(radnoMjesto))
        cout << "Rezervacija dodana" << crt;
    if (!amina.DodajRezervaciju(laboratorija))
        cout << "Rezervacija nije dodana - prekoračen dnevni limit"
<< crt;

    CentarZaRad radniKutak("Radni kutak"), poslovnaZona("Poslovna
zona");
    radniKutak.DodajKorisnika(amina);
    radniKutak.DodajKorisnika(goran);
    poslovnaZona.DodajKorisnika(berun);

    try {
```

```

        //DodajKorisnika onemogućava dodavanje korisnika sa istom
sifrom i baca izuzetak
        radniKutak.DodajKorisnika(amina);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    Rezervacija goranovaSala("SALA-B", SALA, vrijeme4, 120);
    //RegistrujRezervaciju pronalazi korisnika na osnovu sifre i
dodaje mu rezervaciju.
    //i dalje vase pravila definisana u metodi DodajRezervaciju.
metoda vraca true ili false.
    if (radniKutak.RegistrujRezervaciju(goran.GetSifra(),
goranovaSala))
        cout << "Rezervacija registrovana" << crt;

    //AktivniKorisnici vraca pokazivace na korisnike koji imaju
najmanje onoliko rezervacija
    //koliko je definisano vrijednoscu proslijedjenog parametra.
    vector<Korisnik*> aktivni = radniKutak.AktivniKorisnici(1);
    for (auto korisnik : aktivni)
        cout << korisnik->GetImePrezime() << " ima " << korisnik-
>GetRezervacije().size()
        << " rezervacija" << crt;

    //KoristenjePoTipu vraca kolekciju parova (korisnik, broj minuta)
za sve korisnike koji
    //imaju najmanje jednu rezervaciju prostora proslijedjenog tipa.
    Kolekcija<Korisnik, int, 50> koristenjeSala =
radniKutak.KoristenjePoTipu(SALA);
    for (int i = 0; i < koristenjeSala.GetTrenutno(); i++)
        cout << koristenjeSala.GetPrvi(i).GetImePrezime() << " -> "
        << koristenjeSala.GetDrugi(i) << " minuta" << crt;

    vector<CentarZaRad> centri;
    centri.push_back(radniKutak);
    centri.push_back(poslovnaZona);

    /*
Funkcija UcitajPodatke učitava podatke o centrima za rad i
njihovim korisnicima iz
    datoteke čije ime se proslijeđuje kao prvi parametar. Svaka linija
je zapisana u formatu:

        naziv centra|ime i prezime korisnika

Za svaki ispravan red potrebno je:
    - pronaci postojeći ili kreirati novi centar za rad,
    - kreirati i dodati korisnika u odgovarajući centar,
    - onemogućiti dupliranje centara i korisnika unutar istog
centra.

Funkcija vraca true ako je učitano najmanje jedan novi podatak, a
false ako datoteka ne
postoji ili nije učitano nijedan novi podatak.

Primjer sadržaja datoteke:

```

```
Radni kutak|Emina Junuz
Radni kutak|Jasmin Azemovic
Poslovna zona|Zanin Vejzovic
*/
if (UcitajPodatke("korisnici.txt", centri))
    cout << "Ucitavanje uspjesno" << crt;
for (auto& centar : centri)
    cout << centar.GetNaziv() << " sa " <<
centar.GetKorisnici().size() << " korisnika" << crt;

cin.get();
return 0;
}
```